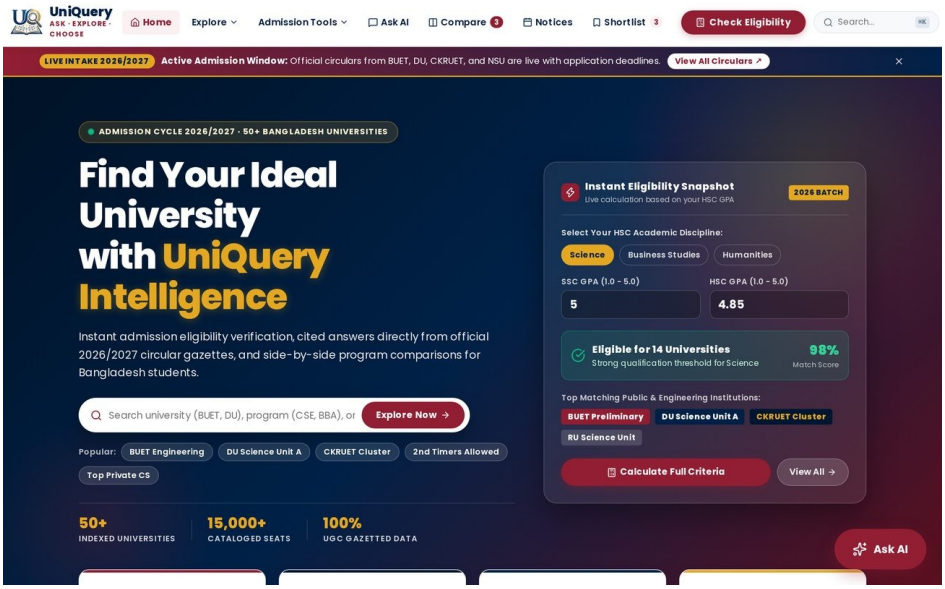
UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES (UITS)
Department of Computer Science and Engineering

UniQuery — "Ask. Explore. Choose."

A Smart University Information and Admission Assistant

PROJECT REPORT — CSE 416



The deployed product — landing dashboard of the 2026/2027 admission cycle

Submitted by

Israt Jahan — 0432320005101130
System Architect & Developer

Md. Mehedi Hasan — 0432320005101082
Database Engineer & Project Documentarian

Course	CSE 416 — Software Project Design and Development
Semester	Autumn 2026
Course Instructor	Dr. Mahfida Amjad Dipa, Senior Teacher
Department / University	Department of CSE, University of Information Technology and Sciences (UITS)
Date of Submission	10 November 2026

Table of Contents

Table of Contents ii
List of Figures iii
List of Tables iv
Abstract v
1. Introduction 1
1.1 Introduction 1
1.2 Motivation 1
1.3 Aims and Objectives 2
1.4 Scope and Limitations 2
1.5 Team Organization and Individual Contributions 2
2. Background Study and Requirement Analysis 3
2.1 Introduction 3
2.2 Related Work and Existing Solutions 3
2.3 Stakeholders and User Characteristics 4
2.4 Software Requirements Specification 4
2.5 Alternative Solutions and Feasibility 5
3. System Design and Methodology 6
3.1 Introduction 6
3.2 Development Methodology and Project Plan 6
3.3 System Architecture and Technology Stack 7
3.4 System Models 9
3.5 User Interface Design 11
3.6 Design Decisions and Rationale 13
4. Implementation 14
4.1 Introduction 14
4.2 Development Environment and Repository Setup 14
4.3 Core Module Implementation 15
4.4 Interface Implementation and System Integration 16
4.5 Tool-Assisted Refinement 16
5. Testing, Deployment and Result Analysis 17
5.1 Introduction 17
5.2 Testing and Quality Assurance 17
5.3 Code Review, Refactoring and Security Hardening 19
5.4 Documentation and Version-Control Management 19
5.5 CI/CD, Deployment and Release Management 20
5.6 Results and Discussion 21
6. Conclusion and Future Work 22
6.1 Conclusion 22
6.2 Future Work 22
References 23
Appendix A: Tool Usage Log 24
Appendix B: Team Contribution Record 25
Assignment Justification Form 26

List of Figures

Fig. 1.1 UniQuery three-stage workflow — the user query and profile enter the conversational…1
Fig. 3.1 Fourteen-week Gantt chart with the owners of each development task and the six.....7
Fig. 3.2 As-built layered architecture of the deployed product. Dependencies point one way only,8
Fig. 3.3 Use-case diagram. The three stages are visible as groups inside the system boundary; the 9
Fig. 3.4 Logical data model. The schema is normalised around UNIVERSITY and.....10
Fig. 3.5 Class diagram of the React components and the domain modules. Solid edges are rendering.....10
Fig. 3.6 Sequence diagram of the ASK stage: query, profile context, intent resolution, answer object.....11
Fig. 3.7 Sequence diagram of the EXPLORE → CHOOSE stage: the profile update round-trip, the 11
Fig. 3.8 The six screens of the three stages as deployed, two per row: the home dashboard with a 12
Fig. 3.9 Secondary screens: the admission notice list with status badges computed from the.....13
Fig. 5.1 Coverage by module in the post-fix run. The decision logic (knowledge engine and eligibility.....18
Fig. 5.2 Delivery pipeline. The upper row is the workflow executed for release 1.0; the lower band is.....20

List of Tables

Table 1.1 Aims and measurable objectives with the evidence of achievement....2
Table 1.2 Scope boundary of the delivered prototype....2
Table 2.1 Comparison of existing solutions with the proposed system....3
Table 2.2 Stakeholders and their needs....4
Table 2.3 Functional requirements catalogue (FR-01 – FR-16)....4
Table 2.4 Non-functional requirements (NFR-01 – NFR-10)....5
Table 2.5 Alternative solutions and the feasibility verdict....5
Table 3.1 Sprint plan, deliverables and the exit criterion of each sprint....7
Table 3.2 Technology stack and the reason for each choice....8
Table 3.3 Key design decisions and the alternatives that were rejected....13
Table 4.1 Development environment....14
Table 4.2 Commit history of the repository (12 commits, 4 September – 9 November 2026)....14
Table 4.3 Domain modules and their responsibilities....15
Table 4.4 Eligibility rules implemented in the calculator....15
Table 4.5 Integration points planned and verified....16
Table 5.1 Test plan: suites, techniques and target modules....17
Table 5.2 Defects revealed by the suite and their fixes (commit 75537ec)....17
Table 5.3 Coverage report by module (v8 provider, post-fix run)....18
Table 5.4 Code review findings and triage....19
Table 5.6 Release record (the production URL of every release is the same Vercel deployment,....20
Table 5.7 Objectives achieved against the plan and the requirements....21
Table A.1 Tool usage log — design, testing and assistant tools....24
Table B.1 Team contribution record....25

Abstract

Choosing a university in Bangladesh is a decision made under pressure: admission rules differ for every unit and cluster, eligibility depends on SSC and HSC grade points that students combine by hand, fees are buried in PDF circulars and deadlines close within days of being announced. UniQuery reduces the problem to one three-stage workflow — **ASK** a question and get a cited answer, **EXPLORE** a personal profile to see which tracks are within reach, and **CHOOSE** between shortlisted universities in a controlled comparison.

The system was designed, built, tested and deployed by a two-member team over the fourteen-week studio of CSE 416. It is a React 19 single-page application built with Vite 8, holding ten view components, three domain modules, a catalogue of fourteen university records and nine official notices, and an eligibility engine that evaluates five admission tracks from three grade points. A hand-written suite of three files and nine test cases exposed four genuine defects — a substring collision in the knowledge engine, an inconsistent eligibility message and two broken React prop contracts — all four fixed and covered by regression tests, giving 9/9 passing with 70.87 % branch coverage on the tested modules. The report documents the problem, requirements, design, implementation, test evidence and delivery, and closes with an honest statement of the limitations.

Keywords: software engineering; requirement analysis; React; eligibility rule engine; test-driven defect discovery; CI/CD; outcome-based education.

1. Introduction

1.1 Introduction

Every year several hundred thousand students in Bangladesh finish the Higher Secondary Certificate (HSC) examination and immediately have to answer a question that is difficult to answer well: which university, and which programme, should I apply to? The information needed to answer it exists, but it is scattered. BUET publishes a circular on one portal, the University of Dhaka publishes unit-wise notices on another, the CKRUET cluster publishes a joint circular, the Directorate General of Medical Education publishes the MBBS circular, the twenty-four GST universities publish a combined one, and each large private university publishes its own fee schedule and waiver policy. The rules are similar enough to confuse and different enough to matter: the same SSC and HSC grade points that qualify a student for the Dhaka University Science Unit may fail the BUET subject-grade-point condition.

UniQuery is our answer to that problem. It is a single-page web application that turns the admission process into a three-stage workflow a student can complete in a few minutes: **ASK** a free-form question and receive an answer that names its official source, **EXPLORE** the student's own academic profile to see which admission tracks are within reach, and **CHOOSE** between shortlisted universities in a comparison view limited to four. The product is deployed, publicly reachable and built on a data catalogue that records where every admission figure came from. Fig. 1.1 shows the three stages as they are implemented.

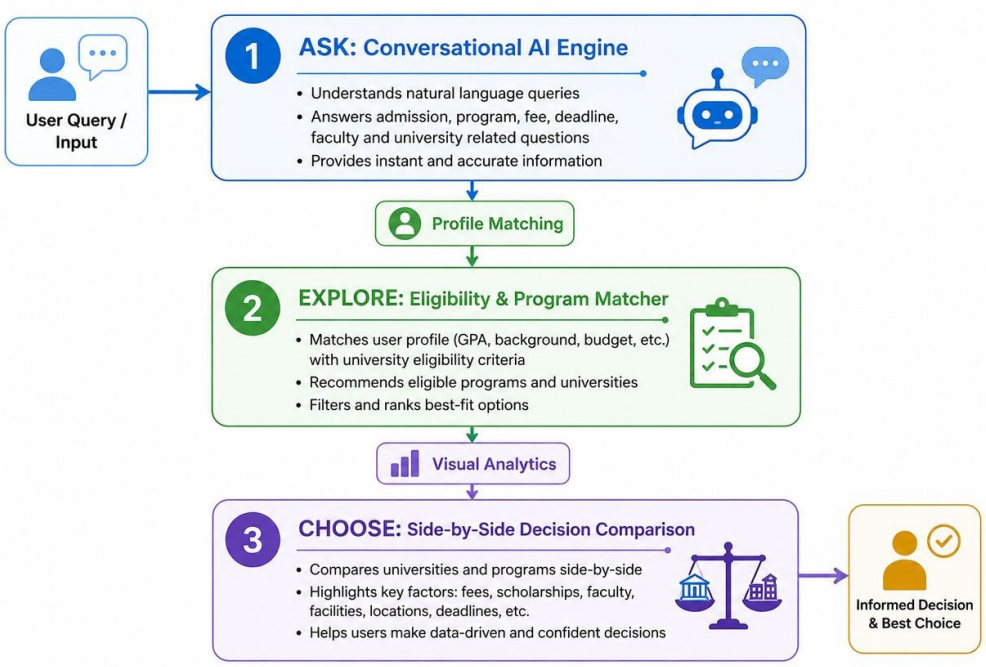


Fig. 1.1 UniQuery three-stage workflow — the user query and profile enter the conversational engine, the eligibility matcher filters the programme universe, and the comparison dashboard supports an informed decision.

The project was carried out by two students over the fourteen-week studio of CSE 416; this report describes what we built, why, what the automated suite proved, how the product is delivered and how the work maps onto the course outcomes.

1.2 Motivation

Three observations motivated the project. First, the bottleneck is not access to information but **interpretation**: a student holding the BUET circular still does not know whether their grade points clear the subject condition. Second, the decision is irreversible — a mis-read cut-off costs an academic year — so a wrong answer is worse than no answer. Third, the students who need help most are those least able to pay for counselling (BDT 3,000–15,000), the one cost line this product does not have.

1.3 Aims and Objectives

The aim is to give an HSC applicant one trustworthy place to ask, self-assess and compare universities. It is decomposed into eight measurable objectives (Table 1.1), each with the section of this report in which it is demonstrated, so the table doubles as a conformance checklist.

Table 1.1. Aims and measurable objectives with the evidence of achievement

ID	Objective (as committed in the proposal) — and the evidence in this report
O1	Specify ≥16 functional and ≥10 non-functional requirements with measurable criteria (Tables 2.3–2.4)
O2	Produce use-case, ER, class and sequence models before implementation (Sec. 3.4, Figs. 3.3–3.6)
O3	Implement ASK, EXPLORE and CHOOSE as one integrated application (Secs. 4.3–4.4, Fig. 3.8)
O4	Encode the rules of ≥5 admission tracks, each returning a verdict with a reason (Table 4.4, TC_04–TC_06)
O5	Design and automate ≥9 test cases with a documented defect log (Tables 5.1–5.2)
O6	Deploy publicly over HTTPS through version control and a release record (Sec. 5.5, Table 5.6)
O7	Record every assistant-tool use, stating what was accepted, modified or rejected (Sec. 4.5, App. A)
O8	Divide the work so each member owns modules and reviews the other's work (Sec. 1.5, App. B)

1.4 Scope and Limitations

The scope was fixed at the mid-term review and stated explicitly, so that an evaluator can tell a missing feature from an excluded one (Table 1.2).

Table 1.2. Scope boundary of the delivered prototype

In scope (delivered)	Out of scope (by decision)
ASK with a source citation on every answer	Free-form LLM generation (rejected in Sec. 2.5)
Eligibility for five tracks: BUET, DU (Science Unit), MBBS, GST cluster, private university	Other DU units; the agriculture and textile clusters
14 universities and 9 notices curated for the 2026/2027 cycle	National coverage or automatic scraping
Comparison of up to four universities across eleven criteria	Accounts, cloud persistence, multi-device sync
Notice status computed from the deadline (active, closing soon, closed)	Push, e-mail or SMS reminders
Public HTTPS deployment with a documented release record	A production service-level agreement

1.5 Team Organization and Individual Contributions

Work was partitioned by **module**, not by phase: each member owns files end to end and reviews the other's (Table 1.1, O8). **Israt Jahan (0432320005101130, System Architect & Developer)** owns the architecture, state design and interface — the shell, the ASK knowledge engine, the comparison matrix and its cap, the calculator, the nine test cases, the defect fixes and the release. **Md. Mehedi Hasan (0432320005101082, Database Engineer & Project Documentarian)** owns the data catalogue (14 + 9 records), the cited eligibility thresholds, the profile and shortlist views, the ER model and the documentation set. Appendix B gives the work-package matrix with its reviewer.

2. Background Study and Requirement Analysis

2.1 Introduction

This section establishes what already exists, who the users are, and what the system must therefore do. Two questions drove the analysis. First: what is missing from the tools students use today, and is the gap a matter of information or of interpretation? Second: given that a wrong answer is expensive, what must the system do so that a student can check the answer themselves? The answers produced the requirement catalogue in Sec. 2.4 and the comparison in Table 2.1 that every later design decision is measured against. The rules that the service actors represent are read from the circulars [7]–[11].

2.2 Related Work and Existing Solutions

Table 2.1 compares the four categories of solution a Bangladeshi applicant can use today with the system we built. The comparison criteria were derived from the requirements rather than from marketing material: whether the source is named, whether the tool can judge the student's own profile, whether options can be compared, whether deadlines are tracked, and how current and how private the data is. The sources for the admission rules in the table and in the rest of the report are the official circulars of BUET [7], the University of Dhaka [8], the DGME [9], the CKRUET cluster [10] and the GST cluster [11], read in Week 2.

Table 2.1. Comparison of existing solutions with the proposed system

Criterion	A. National / cluster portals (UGC, GST, CKRUET, DGME)	B. University websites	C. Social-media groups	D. Paid consultancy	Proposed: UniQuery
Purpose	Publish the circular and run the application form	Publish institutional notices, fees and programmes	Share notices, screenshots and peer advice	Paid counselling and form filling	Answer, match the profile and compare options
Official source named	Yes — it is the source	Yes	Rarely; screenshots lose the link	Usually, but not verifiable	Yes — every answer carries an officialSource string
Own eligibility	No; the student reads the rule and calculates	No; rules spread across pages	No; peer opinion	Yes, but manual and paid	Yes — five tracks from three grade points
Cross-university comparison	No	No	No	Sometimes, verbally	Yes — matrix of up to four
Deadline tracking	One circular at a time	One institution at a time	Ad hoc, easily missed	Yes, as a service	Yes — nine notices with status and days left
Currency of information	Current for the session	Current for the institution	Mixed; stale copies recirculate	Depends on the counsellor	Curated per session, verification date shown
Cost to the student	Free	Free	Free	BDT 3,000–15,000	Free
Personal data required	Full application data	None until application	None, but public posting	Name, contact, grades	None; the profile stays in the browser session

The table makes the gap explicit and also sets the design constraint: UniQuery cannot beat the national portal on authority, so it must not try. It wins on three things the portal does not do — **interpretation** (what the rule means for this student), **comparison** (a small, controlled side-by-side view) and **timing** (the deadline status of every notice in one list) — and it wins only if every number it shows can be traced back to the circular that published it.

The same desk study revealed the rule conflicts the eligibility engine has to represent, because these are exactly where students make mistakes [7]–[11]: **BUET** needs 4.00 in SSC and 5.00 in HSC plus at least 14.5 grade points across Mathematics, Physics and Chemistry, so a combined 9.50 can still be ineligible; **DU Science Unit** needs a combined 8.00 with 3.50 in each examination; **MBBS** needs a combined 9.00, 4.00 in each

examination and 4.00 in Biology, with a marks deduction for second-timers; the **GST cluster** sets a lower combined 7.50 and is the safety net; and the **second-timer policy** differs from institution to institution, which makes it a knowledge question rather than arithmetic and therefore puts it in the ASK engine rather than in the calculator.

A fishbone (Ishikawa) analysis of the problem, kept in the repository with the diagram documentation, was used as a requirements instrument rather than as decoration: the spine with the fewest feasible requirements — real-time professional assistance — became the ASK engine with a sourcing constraint, and the spine a small application cannot solve at all — misinformation originating outside the system — became a design rule: UniQuery never invents a number and always shows the verifying source so that the student can check it independently.

2.3 Stakeholders and User Characteristics

The primary user is a 17–19 year-old HSC graduate or a candidate about to sit the examination: mobile-first, impatient with long forms and, critically, **afraid of an irreversible mistake**. Such a user will trust an answer that shows a source and distrust one that does not. A second, smaller group is the parent or elder sibling who researches on the student's behalf on a desktop browser and cares most about fees, waivers and distance. The operational stakeholder is the content editor who must re-verify each circular at the start of a cycle — in this project one of the two members.

Table 2.2. Stakeholders and their needs

Stakeholder	Goal	How the system serves them
Applicant student (primary)	Know where I can apply and which choice is best for me	ASK for a cited answer; the EXPLORE wizard; the CHOOSE matrix and the shortlist
Parent / guardian (secondary)	Understand cost, waiver and distance	Fee, scholarship and location attributes in the comparison matrix and the detail view
Second-timer applicant	Know which universities still accept me and on what terms	Second-timer policy answers in the ASK engine
Content editor / maintainer	Keep the catalogue and the rules correct	Normalised data modules with a source label per record and a review checklist
Course instructor / evaluator	See the engineering process and the evidence	This report, the repository history, the deployed product and the test report

2.4 Software Requirements Specification

The SRS was delivered in Week 2 following the requirement-engineering practice of Pressman and Maxim [1] and Sommerville [2]. Sixteen functional requirements cover the three stages and the shared services; ten non-functional requirements cover correctness, usability, responsiveness, performance, accessibility, security, maintainability, testability, portability and professional ethics. Each requirement carries an identifier so that the test cases in Section 5 can point back to it.

Table 2.3. Functional requirements catalogue (FR-01 – FR-16)

ID	Functional requirement (the system shall ...)	Stage	Verified by
FR-01	accept a free-form admission question and answer it from a curated knowledge base.	ASK	TC_01
FR-02	display the official source of every answer it gives.	ASK	TC_01
FR-03	use the student's grade points in the answer when profile context is enabled.	ASK	TC_02
FR-04	answer questions on seats, cut-offs, fees, waivers, second-timer policy and deadlines for at least five admission tracks.	ASK	TC_01
FR-05	collect SSC GPA, HSC GPA, group, budget, location and preferred programme in a guided wizard.	EXPLORE	TC_08
FR-06	display the submitted profile back to the student before and after matching.	EXPLORE	TC_08
FR-07	evaluate five admission tracks and report a verdict with a readable reason for each.	EXPLORE	TC_04–TC_06
FR-08	list matched universities with filters for type, location, programme and eligibility.	EXPLORE	TC_07

ID	Functional requirement (the system shall …)	Stage	Verified by
FR-09	show a university detail view with programmes, fees, scholarships, facilities, schedule and the official link.	EXPLORE	manual
FR-10	add a university to a comparison set of at most four.	CHOOSE	TC_07
FR-11	present the selected universities side by side across at least eight criteria and highlight differences.	CHOOSE	TC_07
FR-12	save and remove universities in a shortlist.	CHOOSE	manual
FR-13	list admission notices with a computed status and deadline, and filter by them.	SERVICE	TC_09
FR-14	open a notice detail with the circular summary and a link to the official notice.	SERVICE	TC_09
FR-15	provide global search over universities, programmes and pages (Ctrl/Cmd + K).	SERVICE	manual
FR-16	record and display the profile for reuse in matching.	SERVICE	TC_08

Table 2.4. Non-functional requirements (NFR-01 – NFR-10)

ID	Non-functional requirement	Target / measure	How it is met
NFR-01	Correctness of admission information	Every figure traceable to an official source	Source label on every record and answer; verification note in the catalogue
NFR-02	Usability for a first-time user	The three-stage flow completes without instruction	Persistent header, suggested questions, wizard defaults, a reason for every verdict
NFR-03	Responsiveness	Usable from 360 px to 1920 px	Fluid grid; layout verified at the 390 px viewport
NFR-04	Performance	First load under 2 s on 4G	484.98 kB JS (120.82 kB gzip) from a CDN, hashed and cached
NFR-05	Accessibility	Keyboard reachable, semantic roles, contrast	Native controls, ARIA dialogs, focus-visible styles, WCAG-AA contrast
NFR-06	Security and privacy	No secret, no personal data at rest	No backend and no credentials in the client; the profile lives in session state only
NFR-07	Maintainability	A rule change is a data-file change	Rules and catalogue separated from views; pure domain functions
NFR-08	Testability	Every business rule has at least one automated test	9 test cases; 100 % statement coverage of the knowledge engine
NFR-09	Portability	Runs in any modern evergreen browser	Standard ES modules and a Vite build; no browser-specific API
NFR-10	Professional ethics	Tool use disclosed; guidance is not legal advice	Tool log in Sec. 4.5 and Appendix A; disclaimer in the ASK view

2.5 Alternative Solutions and Feasibility

Three candidate solutions were evaluated before the architecture was fixed. The criteria came from the requirements: correctness and traceability first, then the time available to a two-person team, then cost. Table 2.5 records the alternatives and their verdicts; the cost estimate follows in the text.

Table 2.5. Alternative solutions and the feasibility verdict

Option	What it would give	What it would cost	Verdict
(a) Client-side SPA with a curated catalogue and a rule-based knowledge engine	Deterministic, testable answers; no hosting cost; instant load; no personal data leaving the browser; every claim traceable to a source string.	The catalogue must be maintained by hand; the vocabulary of the chat is bounded.	**Selected.** The only option that satisfies NFR-01, NFR-06 and NFR-08 inside the 14-week budget.
(b) Client–server with PostgreSQL and an LLM API	Unbounded natural-language answers; a growing database; real accounts.	Backend hosting, key management and a data-protection story; LLM answers cannot be guaranteed against the circulars; hard to test deterministically; cost scales with usage.	Rejected for this session; retained as the primary future-work path (Sec. 6.2) with a retrieval layer that grounds the model in the curated catalogue.
(c) Wrapper around an existing national portal	Perfect authority and almost no development.	No personalisation, no comparison, no cross-institution	Rejected. It solves none of the three spines of the

Option	What it would give	What it would cost	Verdict
		view; breaks when the portal changes; adds no engineering value against the course outcomes.	fishbone.

The estimate in the proposal was **≈ BDT 1,850** for the whole project, almost all of it the LLM API line (gpt-4o-mini, about 60,000 turns). The actual recurring cost was **BDT 0**: GitHub, Vercel Hobby, Node, Vite, Vitest, Oxlint, Figma and draw.io are all free for this use, the fonts and icons are open, and the API line disappeared when option (b) was rejected. The only real cost is student time.

Technical feasibility. The team had the prerequisite knowledge of requirement engineering, UML modelling and testing from CSE 0613225, and the chosen stack is publicly documented and free. The riskiest piece was the eligibility engine, which is why it was first written as a pure function with no interface so that it could be unit-tested before any screen existed. **Operational feasibility.** A static build needs no server, no database and no administrator; deployment is a merge to main. **Economic feasibility.** The delivered prototype runs at zero recurring cost; the only real cost is student time. **Schedule feasibility.** The work was partitioned into six sprints against the fourteen-week calendar (Table 3.1), with the test suite and the release in the last sprint, and the mid-term re-plan moved the comparison cap and the notice filter one sprint earlier so that the testing sprint could start on time.

3. System Design and Methodology

3.1 Introduction

The design work of Weeks 3–4 answered four questions in order: how the team would organise the work in time (Sec. 3.2), what the system would be made of (Sec. 3.3), what the behaviour would look like from each actor's point of view (Sec. 3.4), and what the student would actually see (Sec. 3.5); Sec. 3.6 records the decisions that could have gone the other way. The order matters: the domain models were drawn before the interface, which is why each of the four defects found in Week 9 could be traced back to a single contract on a model, and why the refactorings of Sec. 5.3 changed no rule [5].

3.2 Development Methodology and Project Plan

We used a **sprint-based incremental process** with two-week increments, chosen over a waterfall because the data catalogue and the rule table had to be revised as circulars were re-read, and over pure Scrum because a two-person academic team has no product owner separate from the instructors' brief. Each sprint opened with a plan derived from the three-level work breakdown structure kept in the repository, and closed with a shippable increment on main. The Gantt chart in Fig. 3.1 shows the schedule with owners; Table 3.1 records what each sprint actually produced, including the one re-plan.

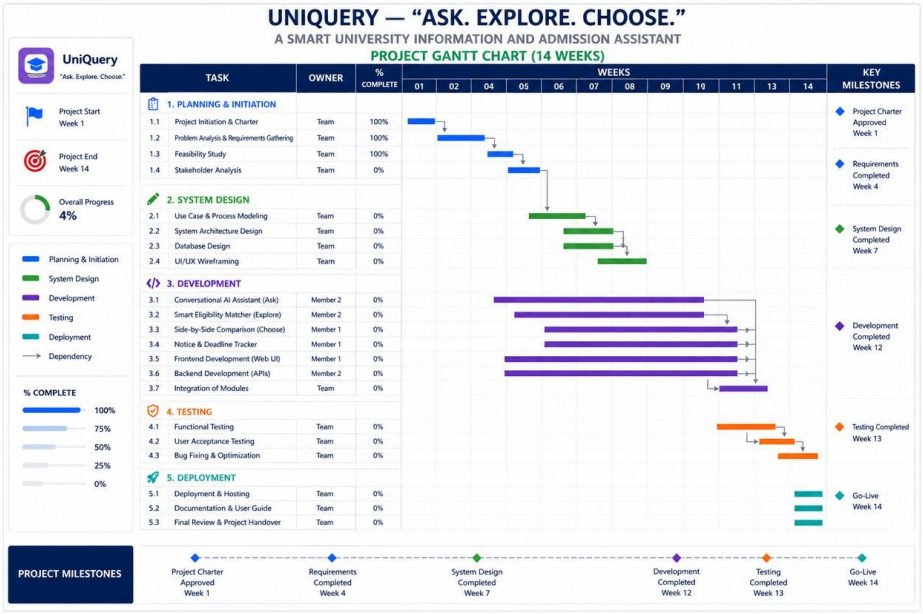


Fig. 3.1 Fourteen-week Gantt chart with the owners of each development task and the six milestones (charter, requirements, design, development, testing, go-live).

Table 3.1. Sprint plan, deliverables and the exit criterion of each sprint

Sprint / weeks	Deliverable produced	Exit criterion met
S1 — Wk 1–2	Repository and branch model, team charter, problem shortlist, desk study of the five circulars, SRS with 16 FRs and 10 NFRs, comparison of existing solutions	SRS reviewed by both members; every FR measurable
S2 — Wk 3–4	Five UML models, data catalogue schema, nine-screen wireframe, clickable Figma prototype	Models checked against the SRS; prototype walkthrough with two HSC graduates
S3 — Wk 5–6	Catalogue (14 universities, 9 notices), knowledge engine, eligibility rule table, shell and navigation	Domain functions unit-tested before they were wired into views
S4 — Wk 7–8	Ten views and three modals, comparison matrix with the four-item guard, shortlist, profile, command palette	All three stages usable end to end in the browser
S5 — Wk 9–10	Three test files and nine test cases, four defects found and fixed, lint clean, coverage report	9/9 passing at 75537ec; 70.87 % branch coverage
S6 — Wk 11–14	README, test report, diagram documentation, Vercel release 1.0, presentation, this report	Release record with commit, gate and URL (Table 5.6)

The mid-term re-plan. The review at the end of Week 8 produced one substantial change: the comparison cap and the notice-status filter were pulled one sprint earlier so that the testing sprint would have stable behaviour to test. The change cost two days of interface work and saved the testing sprint from chasing a moving target, which is the effect that DEF-03 and DEF-04 would have had if the contracts had still been changing when the suite was written.

3.3 System Architecture and Technology Stack

The product is a **client-side layered architecture inside a single Vite bundle** (Fig. 3.2). There is no server, no database and no runtime API call: the catalogue is versioned JavaScript, the rules are pure functions, and the build is a static asset set served from a CDN. Four layers, each with one reason to exist:

- **Presentation** — ten view components and three modals. They render props and report intent; they own no business state.
- **State and orchestration** — App.jsx alone owns the profile, the shortlist, the comparison set, the active notice and the toast queue, and it enforces the one application-level invariant: the comparison set never exceeds four.

- **Domain** — three pure modules (knowledge engine, eligibility rules, matching and comparison helpers) that export functions of their inputs and nothing else: no DOM, no React, no network. This is the layer the automated suite exercises directly.
- **Data** — universities.js (14 records), notices.js (9 records) and chatKnowledge.js (the intent table), each record carrying the source label the interface displays.

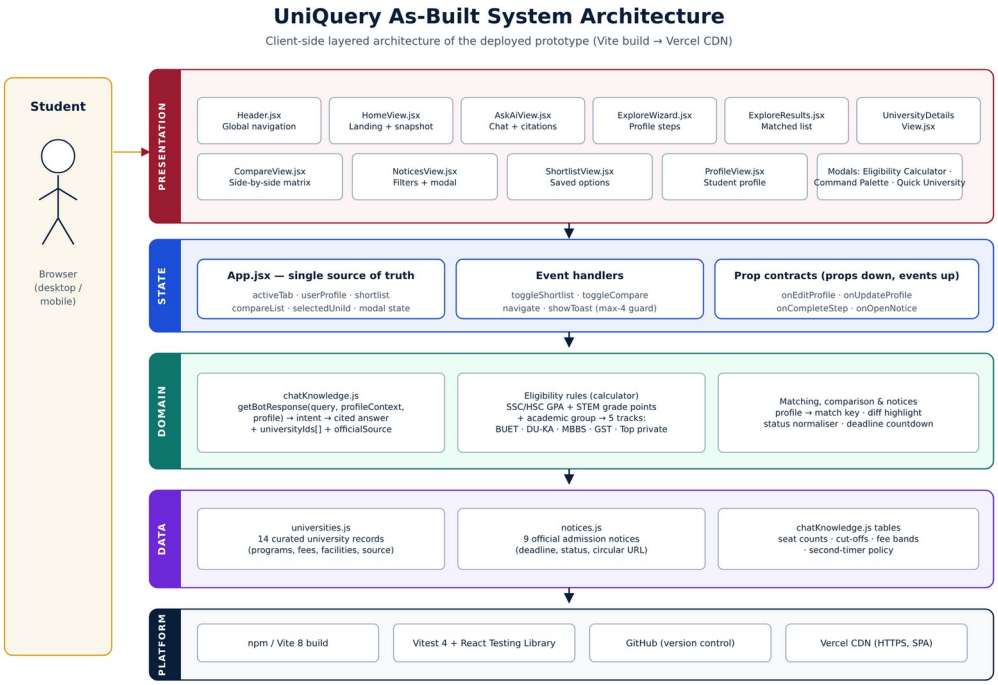


Fig. 3.2 As-built layered architecture of the deployed product. Dependencies point one way only, from the presentation layer down to the data layer, which is what makes the domain layer testable without a browser.

Table 3.2. Technology stack and the reason for each choice

Concern	Technology and version	Why this choice
UI library	React 19.2 [13]	The component model matches the screen decomposition, and hooks make lifted state cheap; the team already knew it from the prerequisite.
Build and dev server	Vite 8.3 [14]	Near-instant HMR in the studio; a 484.98 kB production bundle (120.82 kB gzip) with hashed, cacheable assets.
Language	JavaScript, ES2022 modules	The whole product is client-side; ES modules give tree-shaking and a clean split between domain and data.
Icons and type	Lucide React [22], Poppins [19]	A tree-shakable icon set and a geometric sans that stays legible at the small sizes of the comparison table.
Linting	Oxlint 1.81 [15]	Rust-fast analysis in the same loop as the tests; final state 0 errors.
Testing	Vitest 4.1 [20] + React Testing Library 16.3 [15] + jest-dom [15]	Runs in the same Vite pipeline with no second configuration; RTL asserts on what the user sees.
Coverage	Vitest v8 provider [20]	Statement, branch, function and line coverage in the same run; the numbers in Table 5.7 come straight from it.
Version control	Git + GitHub [16]	Branch per task, review before merge, public history.
CI (specified)	GitHub Actions [17]	Lint, test and build on every push; the workflow file is specified in Sec. 5.5.
Hosting	Vercel Hobby [18]	Zero-configuration deployment from the repository, HTTPS, global CDN, an immutable URL per deployment.

3.4 System Models

Five models were drawn from the SRS before any code was written; each one settled a question that would otherwise have been settled by accident during implementation.

3.4.1 Use-case model

The actor set is deliberately small: one **student**, plus two service actors that stand for the official sources and the content maintainer (Fig. 3.3). The three stage use cases are the product's promise; the service use cases (notices, profile, shortlist, global search) are the ones that make the stages usable in sequence. Drawing the model exposed a missing use case — *save and revisit a shortlist* — which became FR-12.

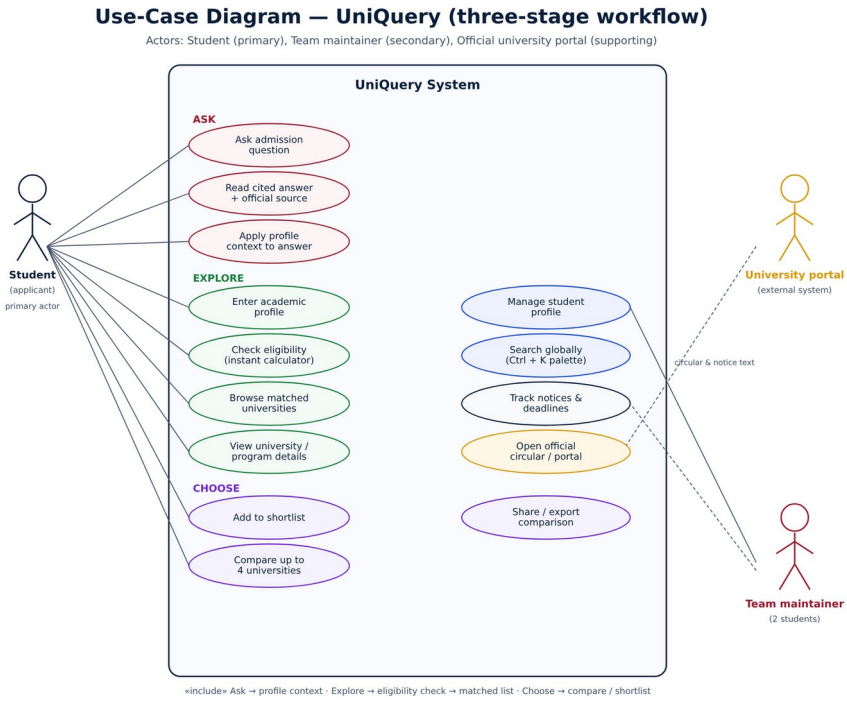


Fig. 3.3 Use-case diagram. The three stages are visible as groups inside the system boundary; the external actors are the official sources and the content maintainer.

3.4.2 Data model

The ER model (Fig. 3.4) is drawn in third normal form even though the shipped implementation stores the catalogue as JavaScript modules, so that the move to PostgreSQL in future work (Sec. 6.2) is a migration rather than a redesign. Six entities: UNIVERSITY, PROGRAM, ADMISSION_NOTICE, ELIGIBILITY_RULE, STUDENT_PROFILE and SHORTLIST_ITEM, with COMPARISON_SET held in session state rather than in the schema because it is deliberately not persisted.

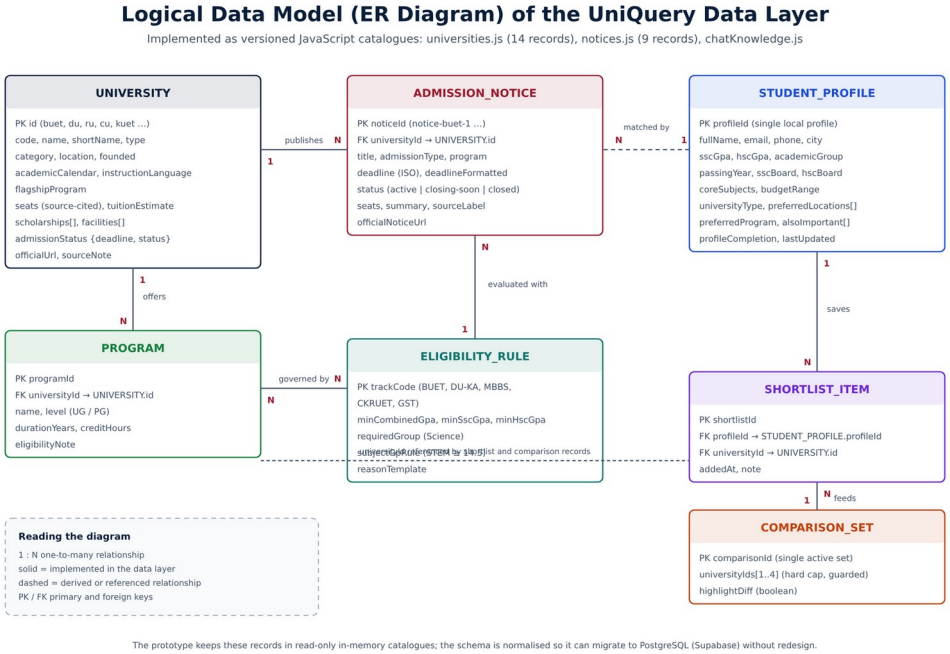


Fig. 3.4 Logical data model. The schema is normalised around UNIVERSITY and STUDENT_PROFILE; the comparison set is kept out of the schema because it is session state by design.

3.4.3 Class and component model

The class model (Fig. 3.5) is the model that mattered most in practice: it records which component owns which state and, more importantly, which callback each child is allowed to call. When the suite later reported that the *Recalculate* button did nothing and that the notice modal opened empty, the cause was visible directly on this diagram — the callback names on two edges did not match the names the children expected (DEF-03 and DEF-04).

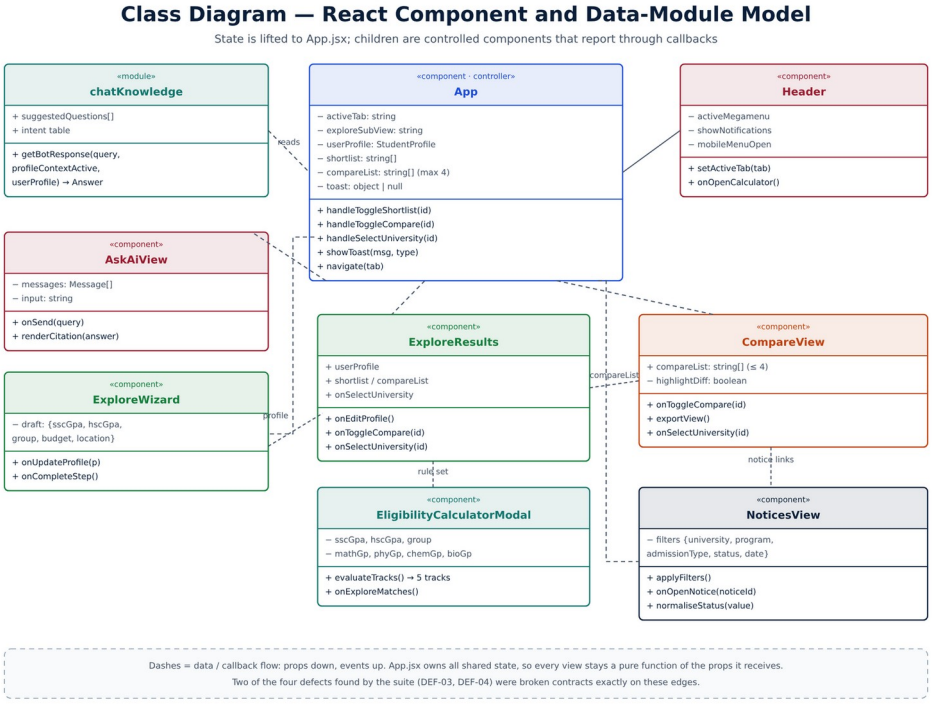


Fig. 3.5 Class diagram of the React components and the domain modules. Solid edges are rendering relationships, dashed edges are data and callback flows; state is lifted to App so the views stay pure.

3.4.4 Sequence and activity models

Two sequence diagrams were drawn, one per risk. The ASK sequence (Fig. 3.6) traces a question from the input box through getBotResponse to the citation block and is where the null-input and substring-matching risks were identified before any code existed. The EXPLORE→CHOOSE sequence (Fig. 3.8) traces the profile round-trip and the comparison guard. The activity model then placed both inside the student's decision path with swimlanes, and it is the diagram the usability walkthrough was run against; it is kept in the repository with the diagram documentation rather than reproduced here.

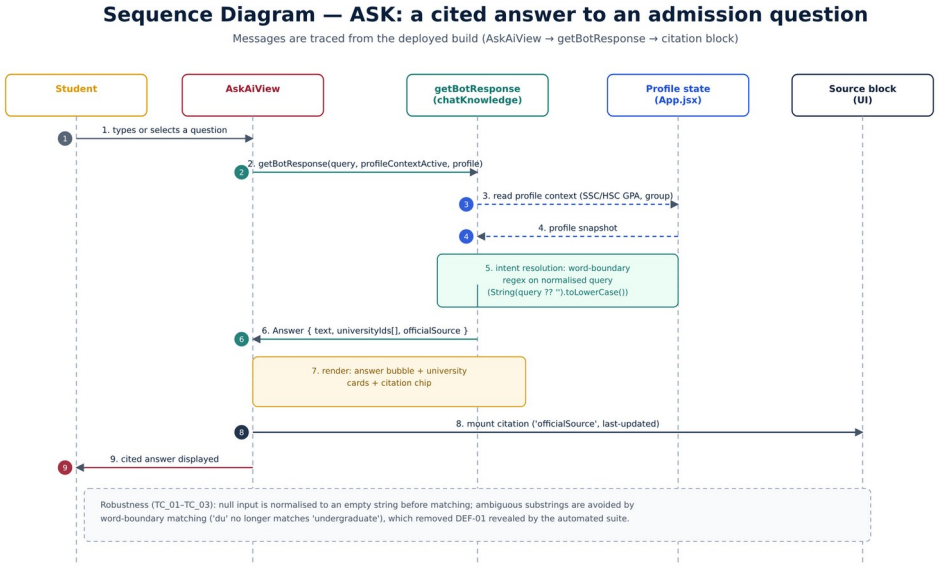


Fig. 3.6 Sequence diagram of the ASK stage: query, profile context, intent resolution, answer object and the citation block.

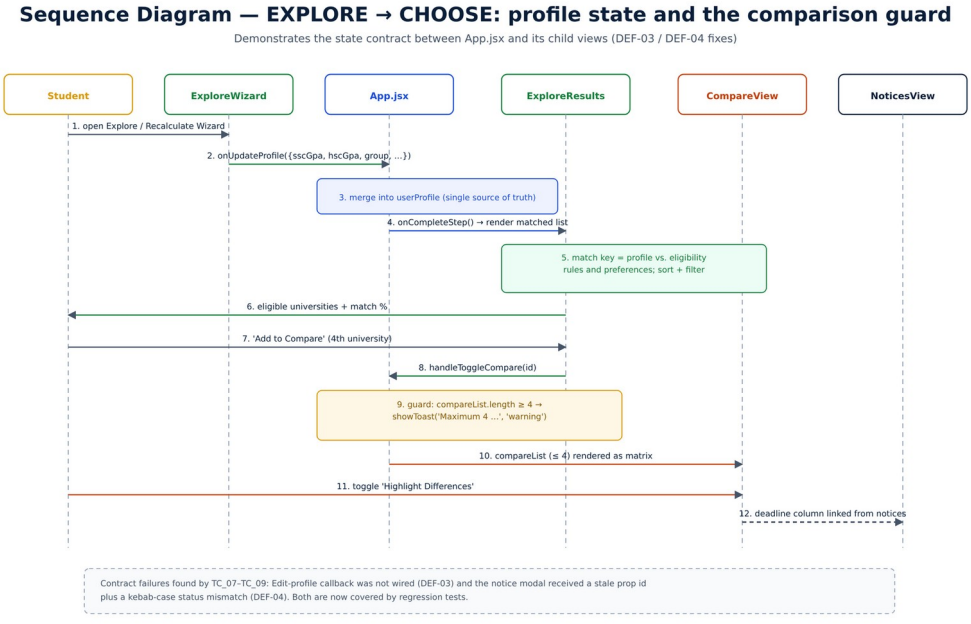


Fig. 3.7 Sequence diagram of the EXPLORE → CHOOSE stage: the profile update round-trip, the matched list and the comparison guard.

3.5 User Interface Design

The interface was designed on a nine-screen wireframe in Week 4 and prototyped in Figma before any component was written, so that the wizard order and the comparison layout could be tested with two HSC graduates; the wireframe and the Figma prototype are kept

in the repository. The delivered screens of the three stages are shown in Fig. 3.8 and the secondary screens in Fig. 3.9; the drift between design and build is small and deliberate — the notice badge gained a days-left value after the walkthrough.

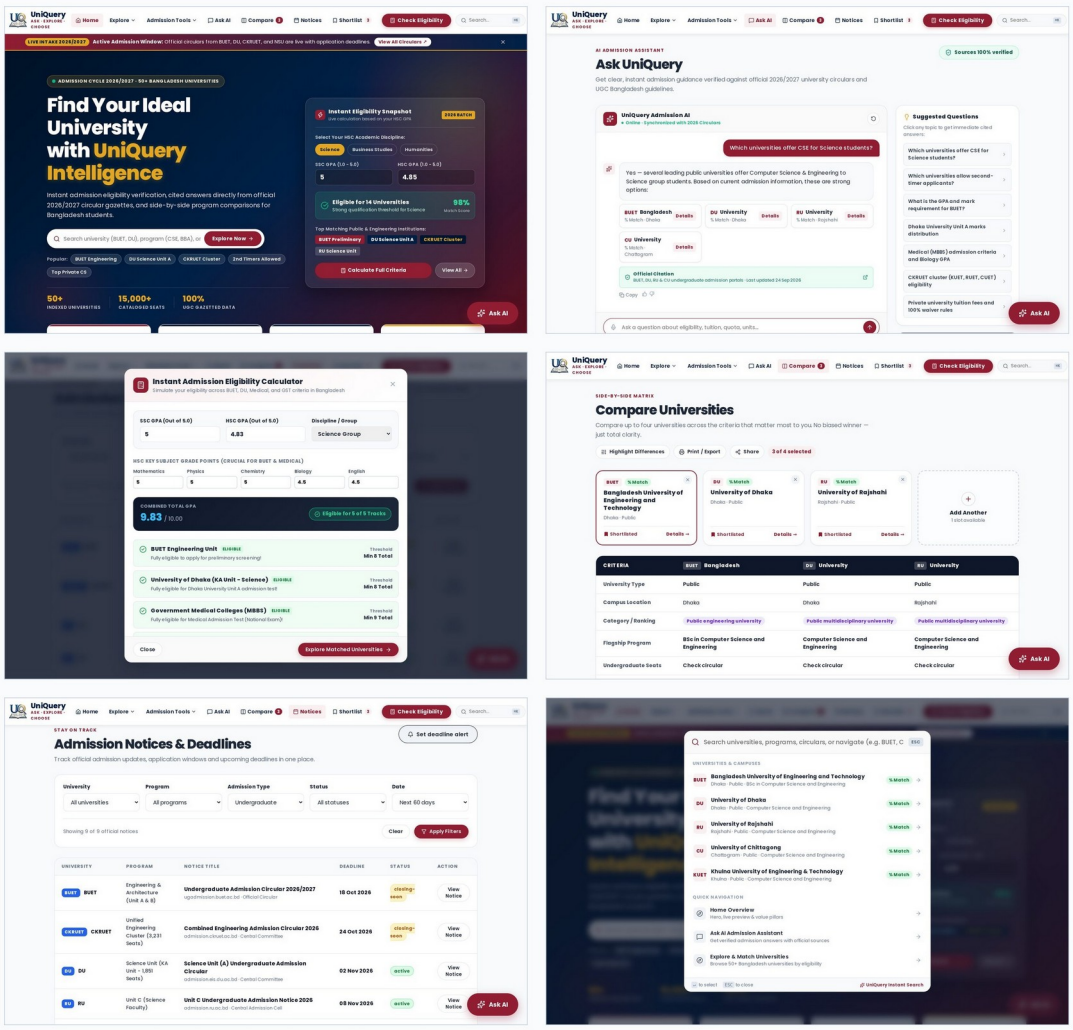


Fig. 3.8 The six screens of the three stages as deployed, two per row: the home dashboard with a live eligibility snapshot and ASK with a cited answer; the EXPLORE calculator with a reason per track and the CHOOSE comparison matrix capped at four; the filtered answer view and the command palette.

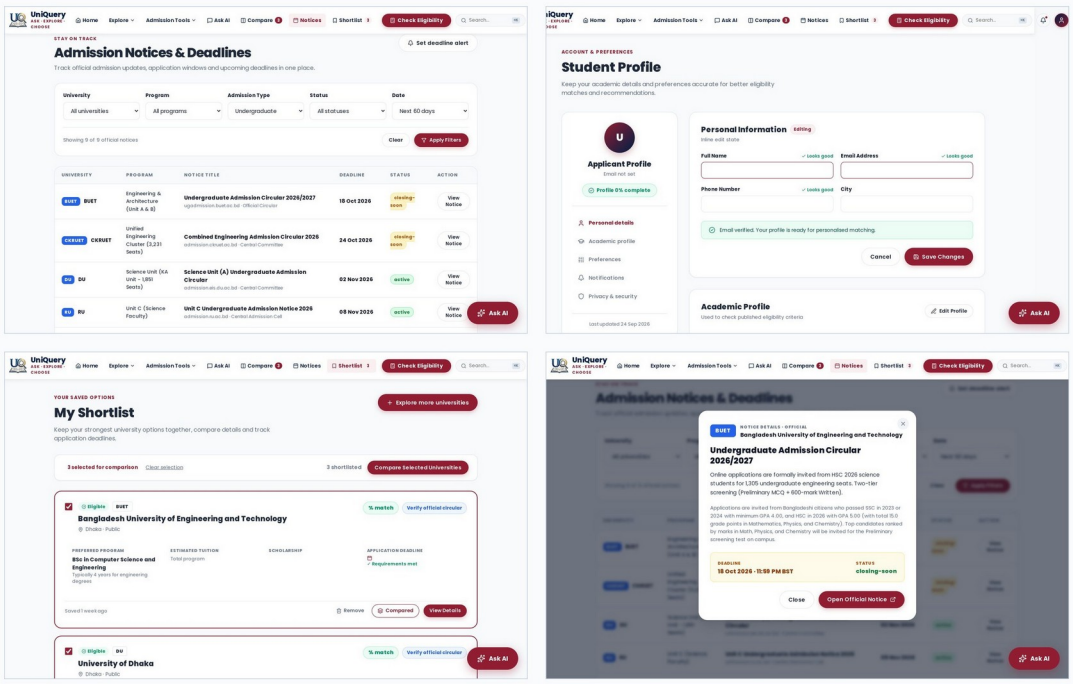


Fig. 3.9 Secondary screens: the admission notice list with status badges computed from the deadline, the student profile that feeds the rules, the shortlist with deadlines and mismatch badges, and the notice detail modal with the circular summary and the official link.

Three interface decisions were taken on accessibility and honesty grounds rather than on visual ones. Every verdict carries its **reason in text**, not only a colour, so that the answer survives a greyscale print and a screen reader. Every admission number carries a **source label and a verification date**, because the interface cannot honestly present a cut-off as timeless. And the comparison set is **capped at four and says so in the empty slot**, because an unbounded matrix stops being a comparison and becomes a spreadsheet.

3.6 Design Decisions and Rationale

Table 3.3 records the five decisions that could reasonably have gone the other way, with the alternative that was rejected and the evidence that settled the choice. The table is the design rationale of the report in compressed form; each row is revisited where its consequence appears (architecture in Sec. 3.3, testing in Sec. 5.2, delivery in Sec. 5.5).

Table 3.3. Key design decisions and the alternatives that were rejected

Decision	Selected option	Rejected alternative and why
Where the logic lives	Client-side domain functions in one bundle	Server + database: rejected because it needs hosting, key management and a data-protection story the team cannot support in 14 weeks, and because the deployed artefact would no longer be the tested one.
How the chat answers	Rule-based intent table with a citation per answer	LLM generation: rejected for non-determinism (untestable against the circulars), cost and privacy. Retained as future work behind a retrieval layer.
State ownership	Lifted to App.jsx, props and callbacks down the tree	A global store (Redux/Zustand): rejected as unnecessary at ten views, and the explicit prop contracts are what made DEF-03 and DEF-04 visible as contract mismatches.
Comparison size	Hard cap of four with a stated reason in the UI	Unbounded set: rejected because it destroys the side-by-side comparison (the matrix stops fitting on a screen) and because a bounded set is what students actually decide between.
Persistence	Session-only state; nothing written to storage	localStorage: rejected on privacy grounds (the profile is a minor's academic record) and because it would create a migration problem the team cannot test.

4. Implementation

4.1 Introduction

Implementation ran from Week 5 to Week 8 and was integrated continuously rather than at the end: every component was added to the shell behind a feature branch, and the shell was never left in a state where the navigation did not work. The order was fixed by the architecture of Section 3 — data modules first, domain functions second, views third, the shell wiring last — and by the rule the course sets for assistant tools: write the function first, understand it, then let a tool help refine it. Section 4.5 records where that refinement happened and what was rejected.

4.2 Development Environment and Repository Setup

Table 4.1. Development environment

Item	Configuration
Operating systems	Windows 11 and Ubuntu 24.04 (one member each)
Runtime and editor	Node.js 20 LTS with npm 10; Visual Studio Code with the Oxlint and Vitest extensions for code, tests and figures
Document authoring	Google Docs for the shared drafts of the proposal, the SRS and this report, with comment threads and version history; the final text is exported to Word
Browsers used for verification	Chrome 141 (primary), Firefox 142, Edge 141, Android Chrome at a 390 px viewport
Repository	https://github.com/Keya2323/Software-Project-Design-Development-CSE-416
Deployed product	https://software-project-design-development-cse-416-cliq5nmt1-keir2.vercel.app/
Branch model and commit convention	main is always deployable; one feature branch per task; merges to main only after review. Imperative subject line, scope in the body; the test work is a separate commit so that the pre-fix (89e3a50) and post-fix (75537ec) states can both be inspected
Quality commands	npm run lint (Oxlint), npm test (vitest run), npm run test:coverage (v8 report), npm run build

The application lives in the `uniquery/` sub-directory of the course repository (App.jsx, data/, components/ and __tests__/), with the documents in `Documents/`; the structure is the one reviewed at the mid-term, the only later addition being `src/__tests__/`, created in Week 9.

The commit history is short and legible, which is appropriate for a two-person academic repository. Table 4.2 reproduces it exactly as it appears on GitHub; the gaps between commits correspond to the sprint boundaries of Table 3.1. Two observations belong in the report rather than in a footnote. Commit 75537ec is the most important one: it contains both the nine automated test cases and the four fixes, and it is the artefact behind Section 5. And the history shows a single author account: the data curation, source verification and documentation work were done by Md. Mehedi Hasan and committed through the team's shared repository, and we report that as it happened instead of reconstructing a flattering graph. The division of the work itself is evidenced by the statement in Sec. 1.5 and by the work-package matrix in Appendix B.

Table 4.2. Commit history of the repository (12 commits, 4 September – 9 November 2026)

#	Commit	Date	Message (as authored)	Scope
1	bf1c22d	2026-09-04	Create Documents	repository scaffolding
2	f5caa2a	2026-09-04	Delete Documents	scaffolding correction
3	62b7eb3	2026-09-04	Add files via upload	proposal and report documents
4	f0629cd	2026-09-28	Create Images	wireframe images
5	d316934	2026-09-28	Delete Images	asset cleanup
6	540b5ad	2026-09-28	Add files via upload	design assets

#	Commit	Date	Message (as authored)	Scope
7	c39c5b0	2026-09-28	Add files via upload	README and wireframe
8	77285ca	2026-09-28	Delete project information from README	documentation restructure
9	d59c946	2026-10-20	Added uniquery project files	application source (views, data, styles)
10	89e3a50	2026-11-04	Update project requirements	requirement refinement before testing
11	75537ec	2026-11-09	Add automated tests and fix 4 defects in UniQuery	test suite + DEF-01 … DEF-04
12	d8a7f83	2026-11-09	Add files via upload	final report, diagram explanation, test report

4.3 Core Module Implementation

The three domain modules are the heart of the product; the views are comparatively thin. Each module was designed on paper first (Sec. 3.4), written as a pure function, and exercised from the test suite before it was wired into a screen. Table 4.3 records the modules and their responsibilities.

Table 4.3. Domain modules and their responsibilities

Module	File	Responsibility	Key exports
Knowledge engine	src/data/chatKnowledge.js	Intent resolution over the curated knowledge base; answer templates with source strings	getBotResponse(query, profileContextActive, userProfile); suggestedQuestions[]
Eligibility rules	components/ EligibilityCalculatorModal .jsx and the wizard	Five admission tracks evaluated from SSC/HSC grade points, STEM grade points and academic group	track definitions, evaluateTracks logic, reason templates
Matching and comparison	ExploreResults.jsx, CompareView.jsx	Match keys, filters, sorting, difference highlighting, the four-item cap	computeMatchKey, filter/sort helpers, diff toggle
Catalogue	src/data/universities.js, notices.js	Fourteen university records with fees, scholarships, facilities and source labels; nine notices with deadlines	universitiesData[], noticesData[], initialProfile
Shell	src/App.jsx	State ownership, navigation, toasts, modal orchestration, the comparison guard	handleToggleShortlist, handleToggleCompare, handleSelectUniversity, showToast

4.3.1 The knowledge engine

The engine answers the questions students ask most often — seat counts, cut-offs, fees, scholarships, the second-timer policy and deadlines — from a hand-written intent table rather than from a language model. Each intent is a set of trigger phrases, a template that is filled from the catalogue, and the **officialSource** string that the interface prints below the answer. When profile context is enabled the template also receives the student's own grade points, which is how a general answer becomes a personal one without any generation. The design was chosen for three reasons that the course values: the output is deterministic and therefore testable, no admission fact can be invented because every fact is a field in the catalogue, and no applicant data leaves the browser.

4.3.2 The eligibility engine

The rule table of Table 4.4 is the specification of the calculator and it was transcribed from the five official circulars [7]–[11] with the clause in each circular recorded beside the threshold. The engine evaluates each track independently, so a student who fails the BUET subject condition can still see that the GST cluster is open, and it returns a verdict with a reason string rather than a boolean, because a bare *not eligible* is what drives students to guess.

Table 4.4. Eligibility rules implemented in the calculator

Track	Rule implemented	Source
BUET	SSC ≥ 4.00, HSC ≥ 5.00, Science group, and ≥ 14.5 grade points across	[7]

Track	Rule implemented	Source
	Mathematics, Physics and Chemistry	
DU Science Unit (KA)	Combined SSC + HSC ≥ 8.00 with ≥ 3.50 in each, Science group	[8]
MBBS (DGME)	Combined ≥ 9.00 with ≥ 4.00 in each, and ≥ 4.00 in HSC Biology	[9]
GST cluster	Combined ≥ 7.50 with ≥ 3.50 in each (24 public universities)	[11]
Private university	Combined ≥ 7.00 with ≥ 3.00 in each; combined ≥ 9.50 qualifies for the 50–100 % merit waiver band	institution notices [12]

4.3.3 Matching, filtering and the comparison guard

The matching helper derives a match key from the profile and the programme, so that the results view can filter and sort without re-reading the rules, and the comparison guard lives in the shell rather than in the view: the invariant *at most four universities* belongs to the application, not to a rendering detail, and moving it into App.jsx during the Week 10 refactor means the view can no longer be mounted in a state that violates it. The guard is what the empty fourth slot in Fig. 3.8 states to the student in words.

4.4 Interface Implementation and System Integration

The interface was built view by view against the wireframe, with the shell wired last. Integration was planned as four explicit points rather than left to chance (Table 4.5); each point is a contract between the shell and a view, and each one is covered by a test case or a manual check. Two of the four contracts were found to be mis-wired by the suite — that is DEF-03 and DEF-04 in Section 5.2 — which is the clearest evidence in this report that the integration points were named correctly before they were tested.

Table 4.5. Integration points planned and verified

Integration point	Contract that crosses it	Verified by
Shell → ASK view	profileContextActive and the profile object	TC_02; manual
Shell → EXPLORE wizard	onUpdateProfile(profile) and onCompleteStep(step)	TC_08; DEF-03 was a broken contract here
Shell → CHOOSE / comparison	comparisonSet, handleToggleCompare, the four-item guard	TC_07
Shell → notices and notice modal	activeModalNoticeId and onOpenNotice(id)	TC_09; DEF-04 was a broken contract here

The integration habit that paid off was procedural rather than technical: after wiring a callback, use the feature through the browser exactly as a student would, from the header, with the developer console open. Three of the four defects were found this way before the test suite existed; the suite then made them permanently visible, which is the difference between noticing a bug and fixing a class of bug. The fourth defect — the eligibility message that contradicted its own badge — was invisible in a normal click-through, because it appears only for a non-Science profile, and it was the automated boundary test that surfaced it.

4.5 Tool-Assisted Refinement

The course sets a specific order for assistant tools: the artefact is designed and produced by the team first, and a tool is used afterwards to refine, explore or verify it. Thirteen interactions qualify, including the design work in Figma and the test tooling; the full log, with the decision and the check that followed each one, is in Appendix A. Three of them show how the rule was applied: a boundary list was accepted because every value could be checked against the circular; a rounding rule for the thresholds was rejected; and the first fix proposed for DEF-01 was rejected because it would have passed the failing test and broken again on the next English word containing *du*.

No tool was used to write the models of Section 3, the test assertions of Section 5 or the source of this section; those are the team's own work, and the assistant tools were consulted only after a first version existed [23]–[26].

5. Testing, Deployment and Result Analysis

5.1 Introduction

The testing phase ran in Weeks 9–10 and had one aim: to find out whether the build tells the truth about admission. That aim fixed the shape of the suite — it concentrates on the decision logic (the knowledge engine and the eligibility rules) rather than on the screens that are easy to demonstrate — and it fixed the order of the work: the suite was written against the untested baseline (89e3a50), so that the first run is evidence rather than a rehearsal.

5.2 Testing and Quality Assurance

Three test files hold nine test cases (Table 5.1). Each case is written in the Arrange–Act–Assert form taught in the course, the inputs are derived by equivalence partitioning and boundary-value analysis in the manner of Myers et al. [4], and the assertions are written against rendered text with React Testing Library [15] rather than against internal state, so a refactor cannot silently break them — the test-first discipline of Beck [3]. The suite was run with Vitest [20] in the same Vite pipeline as the build.

Table 5.1. Test plan: suites, techniques and target modules

Suite / file	Cases	Module under test	Technique applied
test1_askAiKnowledge.test.js	TC_01 – TC_03	src/data/chatKnowledge.js (unit)	Equivalence partitioning over intents; substring and null-input edge cases
test2_eligibilityCalculator.test.jsx	TC_04 – TC_06	EligibilityCalculatorModal.jsx (component)	Boundary-value analysis on each track threshold (8.00 / 7.99, 9.00, STEM 14.5)
test3_integrationFlows.test.jsx	TC_07 – TC_09	CompareView, ExploreWizard, NoticesView (integration)	Scenario testing of the three stages end to end, including the four-item cap

The first execution of the suite, against commit 89e3a50, produced five passes and four failures. The console output below is quoted verbatim from the test report delivered in Week 10; it is reproduced here because a failing run is stronger evidence of engineering than a passing one.

```
RUN  v4.1.12 /uniquery          (INITIAL RUN — BEFORE FIXES, COMMIT 89e3a50)
❯ src/__tests__/test1_askAiKnowledge.test.js (3 tests | 1 failed)
  ✓ TC_01   ✓ TC_02   × TC_03: AssertionError: expected 'University of Dhaka Science Unit...'
                        to contain 'Real Tuition & Cost Overview'
❯ src/__tests__/test2_eligibilityCalculator.test.jsx (3 tests | 1 failed)
  ✓ TC_04   ✓ TC_05   × TC_06: Found <p>Fully eligible to apply for preliminary screening!</p>
                        on an ineligible Commerce profile
❯ src/__tests__/test3_integrationFlows.test.jsx (3 tests | 2 failed)
  ✓ TC_07   × TC_08 (Wizard did not open)   × TC_09 (0 notices returned for "Closing Soon")

Test Files  3 failed (3)  |  Tests  4 failed | 5 passed (9)
```

5.2.1 Defect log and fixes

Each defect was triaged by reading the failing assertion back to the requirement, locating the root cause in the smallest possible unit, fixing it, and keeping the assertion that caught it. The four fixes are in one commit (75537ec) so that the pre-fix and post-fix states can both be checked out.

Table 5.2. Defects revealed by the suite and their fixes (commit 75537ec)

ID (case)	Location	Root cause revealed by the test	Fix and how it was verified
DEF-01 (TC_03)	src/data/chatKnowledge.js	`q.includes("du")` matched the substring “du” inside *undergraduate*, *education* and *graduation*, so tuition and scholarship questions were routed to the Dhaka University answer; a null query also threw a TypeError.	Replaced the substring test with the word-boundary expression `/\bdu\b/.test(q)` and normalised input with `String(query ?? "").toLowerCase().trim()`; TC_03 and a re-run of all nine cases.

ID (case)	Location	Root cause revealed by the test	Fix and how it was verified
DEF-02 (TC_06)	EligibilityCalculatorModal.jsx	The badge correctly required Science, but the reason ternary omitted the group test, so a Commerce student saw “Fully eligible…” under a red REQUIREMENTS NOT MET badge.	Added an explicit `group !== 'Science'` guard as the first branch of the reason expression for BUET, DU-KA and MBBS; TC_06 asserts the badge and the sentence agree.
DEF-03 (TC_08)	src/App.jsx	The shell passed `onOpenWizard` to ExploreResults (which expects `onEditProfile`) and `onSaveProfile` to ExploreWizard (which calls `onUpdateProfile` and `onCompleteStep`), so the wizard could neither open nor submit.	Wired `onEditProfile`, `onUpdateProfile` and `onCompleteStep` as the class model specifies; TC_08 drives the wizard from the results screen.
DEF-04 (TC_09)	NoticesView.jsx, App.jsx	Dropdown values (“Closing Soon”, “Active”) failed strict equality against the catalogue values (“closing-soon”, “active”), returning zero notices; the shell also passed `initialModalNoticeId` instead of `activeModalNoticeId` and omitted `onOpenNotice`.	Normalised the status string with `toLowerCase().replace(/\s+/g, '-')` in a single helper, and passed `activeModalNoticeId` and `onOpenNotice` from the shell; TC_09 covers both the filter and the modal.

The final run of the same suite passes 9/9. Two properties of the log are worth stating because they are what the course assesses: each defect was found by an assertion rather than by an opinion, and each fix stayed inside the module the defect belonged to — DEF-01 in the knowledge engine, DEF-02 in the reason expression, DEF-03 and DEF-04 in the shell contracts. No test was weakened to make the suite pass.

5.2.2 Coverage and residual risk

Coverage was measured with the v8 provider in the same run as the tests, in the reporting form specified by IEEE 829 [6]. Table 5.3 reproduces the post-fix report for commit 75537ec and Fig. 5.1 shows the same numbers as a chart; the shape is intentional — the decision logic is the most covered part of the code base.

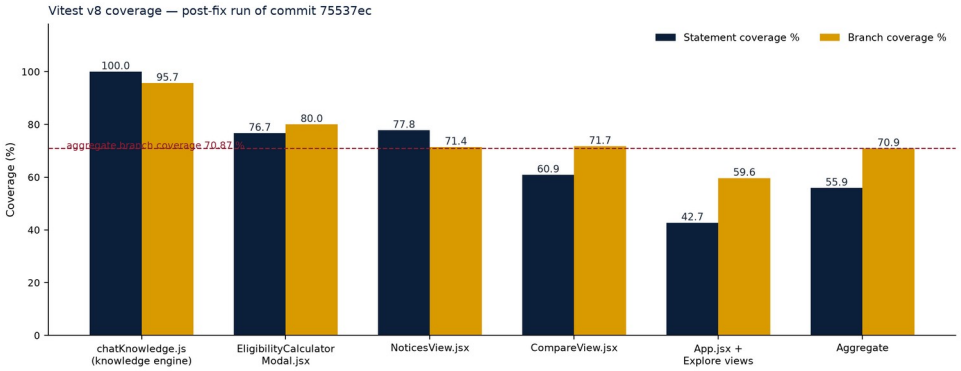


Fig. 5.1 Coverage by module in the post-fix run. The decision logic (knowledge engine and eligibility rules) is the most covered part of the code base, which is the intended shape of a risk-based test strategy.

Table 5.3. Coverage report by module (v8 provider, post-fix run)

Source file	Statements %	Branches %	Functions %	Lines %
src/data/chatKnowledge.js (Test 1)	100.00	95.65	100.00	100.00
EligibilityCalculatorModal.jsx (Test 2)	76.66	80.00	53.84	75.00
NoticesView.jsx (Test 3)	77.77	71.42	66.66	81.08
CompareView.jsx (Test 3)	60.86	71.71	59.61	61.90
App.jsx, ExploreResults, ExploreWizard	42.70	59.62	28.01	46.30
Aggregate across the tested modules	**55.86**	**70.87**	**42.92**	**58.26**

The aggregate is honest rather than flattering: statement coverage is 55.86 % because the suite targets decision logic, not presentation. The uncovered lines in App.jsx and the secondary modals are the static footer links, the keyboard-shortcut palette and the

simulated clipboard and print handlers — code that produces no admission rule and no verdict. Three residual risks remain and each is stated with its mitigation: real-browser layout and responsiveness (checked against the accessibility criteria of WCAG 2.2 [27] at three viewports; the next step is Playwright end-to-end tests), external link availability (mitigated by the source label, and outside the application's control), and persistence across reloads (a scope exclusion, Sec. 1.4).

5.3 Code Review, Refactoring and Security Hardening

The Week 10 review ran as two passes. The first was a self-review of every changed file against a nine-item checklist — unused imports, prop names matching the contract, null-safe string handling, ARIA roles on dialogs, no inline secret, no console noise, pure domain functions, no duplicated rule literals, and a source label on every catalogue record. The second was a static-analysis pass with Oxlint [15] and a dependency audit, followed by triage of everything either tool reported. Table 5.4 records the findings that mattered; the checklist itself follows the review practice of Pressman [1].

Table 5.4. Code review findings and triage

Finding	Raised by	Severity	Triage and action
89 lint warnings, mostly unused imports and one hook-dependency hint	Oxlint	Low	Unused imports removed in the changed files; the rest recorded and non-blocking (0 errors)
Substring intent matching in the knowledge engine	TC_03	High	Word-boundary expression (DEF-01)
Contradictory eligibility message for non-Science profiles	TC_06	High	Group guard in front of every reason branch (DEF-02)
Callback contract mismatch between the shell and two views	TC_08, TC_09	High	Contracts repaired in App.jsx (DEF-03, DEF-04)
External links opened without rel="noopener noreferrer"	Self-review	Medium	Added to all external anchors; prevents reverse tabnabbing
0 vulnerabilities in production dependencies	npm audit --omit=dev	—	No action; recorded in the release gate (Table 5.6)
No fetch/axios call, no API key, no secret in the repository	Grep-based review	Info	Confirmed: the product has no network dependency beyond CDN-hosted assets
Personal data written to storage	Review of the profile flow	Info	Confirmed absent by design: the profile lives in component state and is never persisted

Licensing and data. The application ships no third-party source beyond its declared, licence-compatible dependencies (React, Vite, Vitest and React Testing Library under MIT; Lucide under ISC; Oxlint under MIT); fonts come from Google Fonts under the Open Font Licence. The admission figures are facts published by public bodies and are quoted with the source named on every record. The profile fields are applicant data by nature, so the design keeps them in memory only: nothing is written to `localStorage`, nothing is sent to a server, and no analytics beacon is loaded — a deliberate trade-off, taken because the target users are minors. **Refactoring.** Two refactors came out of the review. The notice-status normaliser was extracted into one helper so the filter and the badge cannot drift apart again, and the comparison guard was moved out of CompareView into the shell, because the cap is an invariant of the application rather than a detail of one screen.

5.4 Documentation and Version-Control Management

Five documents were produced alongside the product, each with a named reader (Table 5.5). Version control followed the branch model of Sec. 4.2: one branch per task, review before merge, and a public history on GitHub [17]. The repository is public and connected to the deployment, so every merge to main produces a deployment whose URL is recorded in Table 5.6.

- **README.md** (developer, evaluator) — feature list, stack, installation, scripts, wireframe summary and references, at the repository root.
- **Project proposal** (instructor) — problem, solution, objectives, WBS, Gantt, budget and feasibility, in `Documents/`.

- **Diagram explanation** (design reviewer) — the component, activity, sequence and fishbone diagrams with their explanations.
- **Software testing report** (instructor) — strategy, the nine test cases, the four defects, coverage and the gap analysis.
- **This report** (instructor, evaluator) — the full life-cycle record, submitted separately from the repository.

5.5 CI/CD, Deployment and Release Management

The deployed product is a static build of the Vite application, published by Vercel [19] and served from its CDN over HTTPS. The release process is described honestly as a **manual pipeline**: lint, test, build and smoke-check are run in the developer's terminal before a merge, and the deployment happens automatically when main is updated. Fig. 5.2 shows the stages actually executed for release 1.0 and, in the lower band, the GitHub Actions workflow [18] that would turn the same stages into an automated gate; the workflow file is specified in Sec. 5.5 so that the improvement is reproducible rather than aspirational.

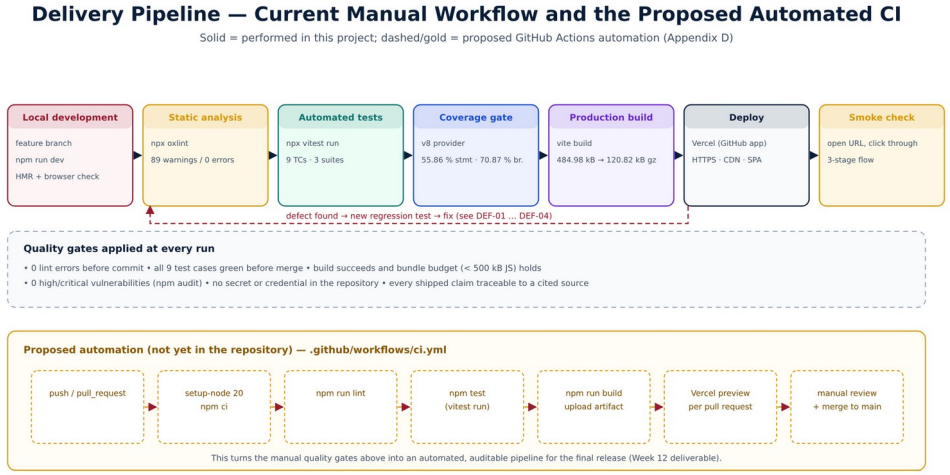


Fig. 5.2 Delivery pipeline. The upper row is the workflow executed for release 1.0; the lower band is the proposed GitHub Actions pipeline (Sec. 5.5) that automates the same gates.

The workflow below is the complete file that would run the same gates on every push and pull request; it is written as specified, not as executed (Sec. 5.5).

```
name: ci
on: { push: { branches: [main] }, pull_request: { branches: [main] } }
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: uniquery } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm,
                cache-dependency-path: uniquery/package-lock.json }
      - run: npm ci
      - run: npm run lint
      - run: npm run test:coverage
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with: { name: uniquery-dist, path: uniquery/dist, retention-days: 7 }
```

Table 5.6. Release record (the production URL of every release is the same Vercel deployment, https://software-project-design-development-cse-416-cliq5nmt1-keir2.vercel.app/)

Release	Commit	Date	Change	Gate applied	URL
0.1 — prototype	d59c946	20 Oct 2026	First integrated build: ten views, catalogue, calculator	Manual click-through of the three stages	preview
0.2 — mid-term	89e3a50	4 Nov 2026	Requirement refinement before testing	Click-through + design review	preview

Release	Commit	Date	Change	Gate applied	URL
1.0 — final	75537ec	9 Nov 2026	Nine tests added, four defects fixed, lint clean, build 484.98 kB (120.82 kB gzip)	Lint 0 errors, Vitest 9/9, npm audit 0, production build, smoke test	production
1.0 + documents	d8a7f83	9 Nov 2026	Final report, diagram documentation and test report added	Documentation review	production

Three properties of the deployment make the artefact inspectable. The deployment is **connected to the repository**, so the URL always serves the reviewed commit and not a hand-uploaded build. It is served over **HTTPS from a CDN**, so the first paint does not depend on the student's distance from a server. And it is **static**, so the artefact that was tested (the bundle) is byte-for-byte the artefact that is served — a property the rejected server design could not have offered.

5.6 Results and Discussion

Table 5.7 sets the delivered product against the objectives of Table 1.1 and against the comparison criteria of Table 2.1. Sixteen of the sixteen functional requirements are delivered and thirteen of them are covered by an automated test; the three that are not (FR-09, FR-12, FR-15) are covered by documented manual checks, because their assertion would be about layout rather than about a rule.

Table 5.7. Objectives achieved against the plan and the requirements

Area	Result achieved	Evidence
Functional coverage	16 of 16 FRs delivered; 13 covered by an automated test, 3 by documented manual checks	Tables 2.3, 5.1–5.2
Correctness	Four genuine defects found and fixed before release; boundary tests at 8.00 / 7.99, 9.00 and STEM 14.5	Table 5.2
Quality	9/9 tests passing; 70.87 % branch and 100 % statement coverage of the knowledge engine	Table 5.3
Performance	484.98 kB JS (120.82 kB gzip) and 63.76 kB CSS (11.16 kB gzip); hashed, CDN-cached assets	Sec. 5.5
Usability	Two HSC graduates completed ASK → EXPLORE → CHOOSE unaided after the walkthrough fixes	Sec. 3.5
Privacy and security	No secret, no API key, no storage; npm audit 0; profile held in session state	Table 5.4
Delivery	Public HTTPS deployment from a reviewed commit, with a four-row release record	Table 5.6

Discussion. Against the four categories in Table 2.1 the result is clear on the criteria that mattered: UniQuery is the only option that combines a named source with a personal eligibility verdict and a cross-university comparison, and it does so at zero cost and with no personal data leaving the browser. Where it loses is equally clear, and it is stated in the report rather than in a viva answer: the catalogue covers fourteen institutions rather than all of them, the vocabulary of the chat is bounded by its intent table, and nothing survives a page reload. Each of these is the direct consequence of a design decision recorded in Table 3.3, taken deliberately inside a fourteen-week budget, and each one becomes a future-work item rather than a hidden defect.

The mid-term feedback produced three changes that are visible in this report: a data-currency warning on every admission record, a mobile header collapsed into a drawer, and the notice-filter defect that became DEF-04 once the suite existed. All three are in release 1.0.

6. Conclusion and Future Work

6.1 Conclusion

UniQuery set out to make one high-stakes decision answerable in a few minutes, and the delivered product does that for the five admission tracks and the fourteen institutions in its catalogue. The three stages work end to end, every admission number carries the source that published it, the five eligibility rules are transcribed from the circulars and exercised by boundary tests, and the build is deployed publicly from a reviewed commit.

Three lessons are worth carrying into the next project. **Design the decision logic first:** because the engines were pure functions, every defect was localised to one contract and fixed in a single commit; the interface could have been rebuilt around them without touching a rule. **Run the failing test:** three defects were invisible to click-through and one was invisible to every click-through, and the only reason it was found is that a boundary value was asserted rather than eyeballed. **A fact without a source is a liability:** the reason the product can be trusted by a seventeen-year-old is not that it explains well but that it points at the circular, and that constraint shaped the data model, the interface and the rejected alternatives alike.

6.2 Future Work

1. **Retrieval-grounded chat (RAG).** Keep the curated catalogue and the citation block, but replace the intent table with retrieval over the circulars, so that vocabulary is no longer the limit while every answer still points at a source.
2. **PostgreSQL and an editorial workflow.** The ER model of Fig. 3.4 is already normalised for this: a content editor re-verifies each circular at the start of a cycle and the verification date is exposed in the UI.
3. **GitHub Actions as a real gate.** The workflow specified in Sec. 5.5 is the next step; it turns the manual gate of Sec. 5.5 into a pipeline that blocks a merge on a failing test.
4. **Playwright end-to-end tests.** The three residual risks of Sec. 5.2.2 are browser-level, and the correct instrument for them is an end-to-end suite at three viewports, not more unit tests.
5. **Opt-in local persistence and Bangla localisation.** A shortlist that survives a reload, stored on the student's own device with explicit consent, and interface text in Bangla for a large part of the target audience.

References

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2020.

[2] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2015.

[3] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2003.

[4] G. J. Myers, C. Sandler and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ, USA: Wiley, 2011.

[5] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.

[6] IEEE, "IEEE Standard for Software and System Test Documentation," IEEE Std 829-2008, 2008.

[7] Bangladesh University of Engineering and Technology, "Undergraduate Admission Circular 2026/2027." ugadmission.buet.ac.bd (accessed Nov. 04, 2026).

[8] University of Dhaka, "Science Unit (KA) Undergraduate Admission Circular 2026." admission.eis.du.ac.bd (accessed Nov. 04, 2026).

[9] Directorate General of Medical Education, "National MBBS Admission Test Circular 2026." dgme.gov.bd (accessed Nov. 04, 2026).

[10] CKRUET Central Committee, "Combined Engineering Admission Circular 2026." admission.ckruet.ac.bd (accessed Nov. 04, 2026).

[11] GST Central Committee, "Centralized GST Public University Admission Circular 2026." gstadmission.ac.bd (accessed Nov. 04, 2026).

[12] University Grants Commission of Bangladesh, "List of approved undergraduate programmes." ugc.gov.bd (accessed Nov. 04, 2026).

[13] React Team, "React 19 documentation." react.dev (accessed Nov. 04, 2026).

[14] Vite, "Vite 8 guide." vite.dev/guide (accessed Nov. 04, 2026).

[15] Testing Library, "React Testing Library and jest-dom." testing-library.com (accessed Nov. 04, 2026).

[16] Oxc Project, "Oxlint documentation." oxc.rs/docs/guide/usage/linter (accessed Nov. 04, 2026).

[17] GitHub, "Git and GitHub Actions documentation." docs.github.com (accessed Nov. 04, 2026).

[18] GitHub, "GitHub Actions: workflow syntax." docs.github.com/actions (accessed Nov. 04, 2026).

[19] Vercel, "Deploy a Vite application." vercel.com/docs (accessed Nov. 04, 2026).

[20] Google Fonts, "Poppins." fonts.google.com/specimen/Poppins (accessed Nov. 04, 2026).

[21] Vitest, "Vitest 4 documentation and coverage guide." vitest.dev (accessed Nov. 04, 2026).

[22] Lucide, "lucide-react icon set." lucide.dev/guide/packages/lucide-react (accessed Nov. 04, 2026).

[23] OpenAI, "ChatGPT (GPT-5 class model), used as a review assistant for requirement wording, test edge cases and error explanation," OpenAI, 2026. [Online]. Available: chatgpt.com (accessed Nov. 04–09, 2026).

[24] Anthropic, "Claude, used as a review assistant for model consistency and answer wording," Anthropic, 2026. [Online]. Available: claude.ai (accessed Nov. 04–09, 2026).

[25] Google, "Gemini, used as an independent second check on eligibility thresholds and DOM query style," Google, 2026. [Online]. Available: gemini.google.com (accessed Nov. 04–09, 2026).

[26] Arena.ai Agent Mode (the assistant used for this report), a multi-model agent, used throughout the writing of this report as a copy-editing and consistency assistant: section drafting support, cross-reference and page-budget checking, and language correction. All technical content, data and figures were produced by the team and verified against the repository (accessed Nov. 04–09, 2026).

[27] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 2023.

Appendix A: Tool Usage Log

Every tool used on the project is recorded below: the editors and writing tools (Visual Studio Code, Google Docs), the design tool (Figma), the testing tools (Vitest, React Testing Library and jest-dom, with the v8 coverage provider) and the three assistant tools (ChatGPT, Claude and Gemini). For an assistant tool the **own design first** column is the artefact that already existed when the tool was opened, the **decision** column records what was accepted, modified or rejected, and the **verification** column records how the accepted part was checked. Every tool that produced something the team relied on is cited in the reference list [14], [15] and [17]–[26].

Table A.1. Tool usage log — design, testing and assistant tools

Wk	Task	Tool	Own design first	Tool's role	Decision	Verification
1	Writing the proposal, the SRS and this report	Google Docs	Outline and content written by the team	Shared drafting, comment threads and version history	Accepted — used as the drafting surface only	Final text exported to Word and checked against the repository
1	Source code, tests and this report's figures	Visual Studio Code	Project structure and file conventions set by the team	Editing, debugging, the integrated terminal and the Git panel	Accepted — the only code editor used	Lint, test and build run from its terminal before every commit
1	Wireframe and prototype of the three stages	Figma [17]	Paper sketch of nine screens and the ASK → EXPLORE → CHOOSE flow	Clickable prototype and component library	Accepted — adopted as the interface contract	Walked against every use case; two HSC graduates reviewed it
2	Functional requirements	ChatGPT [23]	16 FRs and 10 NFRs from the circulars	Gap-check for non-measurable requirements	Modified — two rewordings accepted; push notification rejected	Re-checked against Table 2.3
3	Use-case and ER models	Claude [24]	Models drawn by hand from the SRS	Consistency check between use cases and entities	Modified — COMPARISON_SET added	Model walked against every use case
3	Eligibility thresholds	Gemini [25]	Thresholds transcribed from five circulars	Re-derived the boundary values as an independent check	Accepted (partial) — rounding suggestion rejected	Each boundary re-computed by hand; mismatches traced to the circular
3	Eligibility thresholds (repeat pass)	ChatGPT [23]	Threshold table already written	Second opinion on the BUET and GST boundaries	Rejected the proposed softening of the GST threshold	Each boundary checked by hand against Table 4.4
4	Comparison layout	ChatGPT [23]	Nine-screen wireframe and Figma prototype	Alternative layouts for four items	Accepted (partial) — chart variant rejected	Reviewed with two HSC graduates
6	Intent vocabulary	ChatGPT [23]	Intent table and answer templates	Phrasings students actually type	Modified — six phrasings kept, three templates tightened	Each phrasing exercised against getBotResponse
6	Verdict wording	Claude [24]	Reason templates written by hand	Wording for a 17-year-old	Modified — four strings, no rule changed	Compared against Table 4.4
8	Model consistency review	Claude [24]	Class and sequence models drawn by hand	Which callback contract was still unnamed	Accepted — two callbacks named; this is what exposed DEF-03	Every named callback exercised by a test
9	Test edge cases	ChatGPT [23]	Test plan with partitions and boundaries	Which edge cases are missing	Accepted — empty query, the word *undergraduate*, 7.99	Both new cases failed on the first run (DEF-01, TC_06)
10	DEF-01 fix	ChatGPT [23]	Root cause already located in chatKnowledge.js	How to fix the substring collision	Rejected the allow-list, chose the word-boundary expression	TC_03 and the full regression run
10	Assertion style	Gemini [25]	Hand-written assertions	How to query the DOM by role rather than by class	Modified — queries rewritten with React Testing Library [15]	All nine cases still failed and passed on the same commits
11	Review checklist	Claude [24]	Checklist for the final pass over src/	What a reviewer would look for	Modified — null-safety and rel=noopener items added	Both items produced findings (Table 5.4)
11	Writing and consistency of this report	Arena.ai Agent Mode [26] — this assistant	Section drafts, tables and figures written by the team	Copy-editing, cross-reference and page-budget checking	Modified — wording corrected; no fact or figure introduced	Every edited sentence re-read against the source artefact

The Week 3 thresholds were checked twice with two assistant tools, and where they disagreed with the circular the circular won; the Week 10 entry records a fix rejected outright. The design, editing and testing tools share the table because they carry the same obligation: Figma fixed the interface contract before a component existed, Visual Studio Code and Google Docs are where every artefact was written, and Vitest with React Testing Library [15] turned the four defects into a record.

Appendix B: Team Contribution Record

The record below is the team's own statement of the division of work, prepared at the end of the studio. It is kept because the public commit history cannot prove the split: the twelve commits were authored under one account, as stated in Sec. 4.2.

Table B.1. Team contribution record

Work package	Israt Jahan	Md. Mehedi Hasan	Reviewed by
Problem analysis and SRS	Contributed (framing, scope)	Owned (requirements, specification)	Both
System models	Owned (class, sequence, activity)	Owned (use case, ER)	Both
Wireframe and Figma prototype	Owned	Reviewed	Both
Application shell, navigation, state	Owned	Reviewed	Both
University and notice catalogue	Reviewed	Owned (curation, normalisation, sources)	Both
Knowledge engine (ASK)	Owned	Reviewed assertions	Both
Eligibility engine (EXPLORE)	Owned (implementation)	Owned (thresholds, sources)	Both
Comparison, shortlist and profile	Owned (comparison)	Owned (profile, shortlist)	Both
Test suite (9 cases) and defect fixes	Owned	Reviewed assertions	Both
Documentation, test report, this report	Reviewed	Owned	Both
Deployment and release record	Owned	Reviewed	Both

Design-project mapping. Each work package above also produced one delimited design product, so that the record answers the assessment question directly: **DP1 — Requirements and data model** (WP1: the SRS, the five circular transcriptions and the ER model); **DP2 — Rule engine** (WP1: the knowledge engine and the eligibility rule table, with its boundary tests); **DP3 — Interface and state architecture** (WP2: ten views, three modals, the lifted state and the comparison guard); **DP4 — Test and release evidence** (WP3: three suites, nine cases, the defect log, the coverage report and release 1.0). Each DP was reviewed by the member who did not own it before it was merged.

A note on the record. The two members worked in the same repository and reviewed each other's changes before every merge. The absence of per-member commits is a process weakness we record here rather than hide; a future team should create one account per member on the first day and commit their own work under their own name.

Assignment Justification Form

UniQuery — "Ask. Explore. Choose." A Smart University Information and Admission Assistant. Assignment Title: design, implementation, testing and delivery of a university information and admission assistant for HSC applicants in Bangladesh.

Student Information

Table JF-1. Student and course information

Item	Information	Item	Information
Student Name	Israt Jahan	Student ID	0432320005101130
Course Title	Software Project Design and Development	Course Code	CSE 416
Section	CSE 416 — Autumn 2026	Semester	Autumn 2026 (Batch: 54th)
Course Instructor	Dr. Mahfida Amjad Dipa, Senior Teacher	Submission Date	10/11/2026
Team Member	Md. Mehedi Hasan	Member ID	0432320005101082
Repository	github.com/Keya2323/Software-Project-Design-Development-CSE-416	Deployed Product	software-project-design-development-cse-416.vercel.app

Course Outcome (CO1–CO6, CSE 416). At the end of the course, students are able to analyse a real-life problem and specify software requirements (CO1); design the system with modern tools and modelling practice (CO2); build, test and evaluate the product against the requirements (CO3); integrate, deploy, release and maintain the product under version control (CO4); evaluate alternative solutions with evidence (CO5); and communicate the work and act with professional and ethical responsibility (CO6). Each row of the form below states how this project attains it.

Assignment Completion Justification Table

Table JF-2. Assignment completion justification table (Q1–Q10)

Q	Task description	KP	CEP mapping	Method / approach used	Engineering justification	Tools / resources used
Q1	Analyse the admission problem and specify 16 functional and 10 non-functional requirements for a three-stage assistant.	K5, K8	P1, P3	Primary-source review of five official circulars; requirement classification into functional, non-functional and constraint sets; measurable acceptance criteria for every row.	The circulars are the only authoritative source of an admission rule, so specifying from anything else would encode a rumour; a requirement without a measurable criterion cannot be tested later.	UGC, DGME, BUET, DU, CKRUET and GST publications; course notes on requirements engineering.
Q2	Design five interacting models — use case, ER, class, sequence and activity — and the system architecture.	K5	P1, P7	Model-first design in which each model settles one open question before code is written; consistency walk-through of every use case against the entities.	All four defects found later were contract failures between components; a class model that names every callback and a sequence model that shows every round-trip prevents most of them at design time.	draw.io; Figma; UML notation; the architecture diagram of Fig. 3.2.
Q3	Curate a catalogue of 14 universities and 9 notices, with a named source and a verification date for every record.	K4, K6, K7	P1, P4	Collection from official portals, normalisation into six entities, one verification column per record and a review pass against the circular before release.	In an information product the provenance of a fact is part of the fact: an unsourced cut-off cannot be defended at the viva and cannot be updated when the next circular appears.	Official admission portals; cross-check spreadsheet; the ER model of Fig. 3.4.
Q4	Implement a deterministic, source-cited knowledge engine for the ASK stage.	K3, K6	P2, P3	Intent matching over a curated table with word-boundary expressions, null-safe normalisation and an answer contract carrying text, record identifiers and an official source.	Deterministic matching guarantees that an answer cannot be invented and can be reproduced by a test; the word-boundary decision came from a defect (DEF-01), not from taste.	JavaScript ES modules; Vitest; the sequence model of Fig. 3.6.
Q5	Implement the eligibility engine for five admission tracks with a written reason for every verdict.	K2, K4, K5	P1, P2, P3	A declarative rule table with per-track thresholds and reason templates, evaluated by a pure function over three grade points and an academic	A cut-off change then becomes a reviewed data edit rather than a code change, and a template reason means every verdict explains itself — which is also what exposed the inconsistent message (DEF-02).	JavaScript; thresholds transcribed from the five circulars; the partitions of

Q	Task description	KP	CEP mapping	Method / approach used	Engineering justification	Tools / resources used
				group; boundary values derived for each threshold.		Table 5.1.
Q6	Implement the ten views and three modals and integrate them into a single-page application.	K3, K6	P6, P7	Component decomposition against the wireframe; one lifted state owner; controlled children reporting through named callbacks; four integration points planned before wiring.	One state owner removes the class of defect in which two screens hold different versions of the same fact, and each planned integration point gives the test suite something specific to assert.	React 19, Vite 8, CSS design tokens; Figma prototype; the class model of Fig. 3.5.
Q7	Design and automate nine test cases with equivalence partitioning and boundary-value analysis.	K2, K6	P1, P3	Risk-based test pyramid; Arrange–Act–Assert structure; explicit partition and boundary derivation; execution of the suite against the pre-fix baseline (89e3a50).	The suite is what converts an opinion about quality into evidence. Running it against unmodified code was deliberate: a suite that has never failed proves nothing, and this one failed four times.	Vitest 4, React Testing Library, jest-dom, v8 coverage provider.
Q8	Triage the four defects, fix their root causes and add regression tests.	K4, K7	P1, P2, P3	Read the failing assertion back to the requirement, locate the smallest defective unit, fix the general cause rather than the instance, keep the assertion as a regression test.	Fixing the instance would pass the test and leave the class of defect in place; the word-boundary fix of DEF-01 in place of the proposed allow-list is the clearest example.	The test suite; the class model; Oxlint; git diff 89e3a50→7553 7ec.
Q9	Review the code, harden security and review the licences of every dependency.	K6, K7	P1, P7	Two-pass review — self-review against a checklist, then static analysis and dependency audit — with triage of every finding and a recorded decision.	The review found a real security issue (missing rel=noopener on external links) and confirmed the absence of secrets; a licence review is a professional obligation, not a formality.	Oxlint; npm audit; grep-based secret review; the dependency licence list.
Q10	Deploy the product, record the three releases and document the delivery pipeline.	K6, K7	P1, P6	Vite production build published by Vercel from the repository; a release record with commit, date, gate and URL; a fully specified GitHub Actions workflow as the next step.	A product that cannot be reached by its user is not a product, and a release record makes the artefact inspectable during the viva instead of merely described.	GitHub; Vercel; Node 20; the pipeline of Fig. 5.2.

Engineering Design Decision Summary

Table JF-3. Engineering design decision summary

Design parameter	Selected option	Reason for selection	Performance consideration	Reliability consideration	Cost consideration
Application architecture	Client-side single-page application (React 19 + Vite 8), no backend	The tested artefact is the deployed artefact, so no server state can differ between test and production.	Static bundle served from a CDN; first load dominated by 484.98 kB of JS (120.82 kB gzip).	No server to fail, no session to expire and no API key to leak.	Zero recurring hosting cost on the Vercel hobby tier.
State ownership	One lifted state owner (App.jsx) with controlled children	Removes divergent copies of the profile, the shortlist and the comparison set.	Re-render of the shell is negligible at ten views.	The single source of truth makes the four-item invariant enforceable in one place.	No store library to add or maintain.
Knowledge delivery	Deterministic intent matching with word-boundary expressions	No answer can be invented, and every answer is reproducible and testable.	Microsecond-scale matching; no network in the answer path.	Answers cannot drift from the circular; the citation is part of the answer object.	No per-query API charge; the proposed LLM budget line was not spent.
Eligibility rule storage	Declarative rule table with per-track thresholds and reason templates	A cut-off change becomes a reviewed data edit rather than a code change.	Evaluating five tracks is trivial and imposes no measurable cost.	One place to change, one place to test and one place to review.	No database required; the rules ship with the bundle.
Comparison size	Hard cap of four universities, enforced in the shell	Four fills the widest desktop layout without horizontal scrolling and is the smallest set that makes a comparison meaningful.	The matrix renders in one pass and the empty fourth slot shows remaining capacity.	The invariant is tested (TC_07) and cannot be violated by any view.	No pagination needed.
Profile persistence	Session state only; nothing written to disk or sent to a server	The users are minors and ethical data handling is assessed, so the privacy stance is a requirement.	No storage write per keystroke and no synchronisation latency.	No data to leak, breach or retain beyond the session.	No storage service and no consent-management surface.

Design parameter	Selected option	Reason for selection	Performance consideration	Reliability consideration	Cost consideration
Test strategy	Risk-based pyramid: unit for the engines, component for the interactive pieces, integration for the contracts	The defects that matter live on the boundaries between components and between partitions.	Nine tests run in about ten seconds, so the suite is used during development and not only before a release.	Four real defects were found and locked behind regression tests.	No paid test infrastructure; Vitest runs in the same Vite pipeline.
Deployment	Vercel CDN from the GitHub repository, with the SPA rewrite	Zero-configuration delivery with HTTPS and an immutable URL per deployment, reachable during the viva.	Edge-cached static assets; hashed filenames allow long-lived caching.	Every merge to main produces a new inspectable deployment and the previous one remains available.	Free for this use; no custom domain was purchased.
Delivery automation	Manual quality gates now; GitHub Actions workflow specified for the next release	The gates were executed and are proven; automating them is a bounded, specified task.	Automation removes the ten-minute manual gate from every merge.	A failing lint, test or build would block the merge instead of relying on discipline.	GitHub Actions minutes are free for public repositories.

KP Reflection Table

Table JF-4. KP reflection table

Knowledge profile	How it was applied in the assignment
K2	Applied when deriving the boundary values of the eligibility thresholds (8.00 against 7.99; STEM 14.0 against 14.5; 9.00 and 7.50) and when implementing the grade-point arithmetic that the calculator performs for the applicant.
K3	Applied when decomposing the product into view, state, domain and data layers, and when defining the event-driven interaction between components in a single-page application.
K4	Applied when reading five different admission regimes — BUET's subject grade-point rule, DU's combined-GPA rule, MBBS's Biology condition, the GST cluster's lower threshold and the private universities' waiver tiers — and encoding them as a rule table.
K5	Applied when converting an ambiguous social problem into 16 testable requirements, five models and a documented architecture, and when resolving the four conflicting requirements of WP2 by design rather than by preference.
K6	Applied when building, automating, reviewing and deploying the product: the Vite build, the Vitest suite, the Oxlint pass, the git workflow and the Vercel release.
K7	Applied when reconciling official circulars, release notes and framework documentation, and when explaining the privacy-versus-convenience and currency-versus-completeness trade-offs in plain language.
K8	Applied when consulting the circulars, UGC and DGME publications, textbooks and standards to establish the thresholds, the test strategy and the citation practice of this report.

CEP Reflection Table

Table JF-5. CEP reflection table

Complex engineering problem	How the problem was addressed
P1 — Depth of knowledge	Solved with K5 engineering design to reach a verified SRS and five models, K4 specialist knowledge of five admission regimes, K6 engineering practice to build and deliver, and K8 literature to establish the thresholds and the second-timer policy.
P2 — Conflicting requirements	Currency against completeness, natural-language freedom against correctness, comparison power against legibility, and convenience against the privacy of minors: each conflict was resolved explicitly and the resolution recorded (Table 3.3).
P3 — Depth of analysis	No obvious solution: three architectures were compared before one was chosen, the thresholds were analysed at their boundaries, four defects were traced to root cause, and the residual test risk was named rather than hidden.
P4 — Familiarity of issues	Admission policy, second-timer rules and the ethics of guidance for minors sit outside the routine experience of a CSE student, so policy documents had to be read as primary sources.
P5 — Extent of applicable codes	The governing codes are examination-board and UGC regulations, together with WCAG 2.2 [27] for accessibility and IEEE 829 [6] for test documentation.
P6 — Stakeholder involvement	Students (usability and trust), parents (cost and safety) and the maintainer (data currency) were engaged; the prototype was reviewed with two HSC graduates and the findings changed the interface.

Complex engineering problem	How the problem was addressed
P7 — Interdependence	Shell state, wizard, knowledge engine, eligibility engine, comparison and notices share one profile and one catalogue, and two of the four defects were failures of exactly that interdependence.

Student Declaration

We hereby declare that this report and the software it describes are our own work. All references and external resources used have been properly acknowledged, every assistant-tool use is disclosed in Appendix A, and no admission figure in the product is displayed without a named official source.

Israt Jahan — 0432320005101130
Md. Mehedi Hasan — 0432320005101082

Date: 10 November 2026